

Министерство науки и высшего образования Российской
Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
**«Национальный исследовательский университет
ИТМО»**

Курсовая работа
по дисциплине «Объектно-ориентированное
программирование»

Учет внутриофисных расходов

Выполнил:
студент группы _____

Проверил:

Санкт-Петербург
2024

Оглавление

Глава 1

Введение

Целью данной курсовой работы является разработка приложения на языке Python с графическим интерфейсом PyQt5 для автоматизации учёта внутри-офисных расходов частной фирмы. Приложение реализует объектно-ориентированный подход к моделированию предметной области и поддерживает сохранение/загрузку данных в формате XML.

Предметная область включает три сущности:

- **Отделы** (Название, Количество сотрудников)
- **Виды расходов** (Название, Описание, Предельная норма)
- **Расходы** (Вид расхода, Отдел, Сумма, Дата)

Приложение построено по многослойной архитектуре: слой данных (классы-сущности и списки), слой хранения (чтение/запись XML), слой графического интерфейса (таблицы, формы редактирования, страницы с вкладками).

Глава 2

Задание 1: Реализация классов сущностей

2.1 Пояснение к выполнению Задания 1

Согласно предоставленным источникам, первым этапом разработки является создание базовых классов, моделирующих сущности предметной области. В основе иерархии лежит класс `general`, который обрабатывает общие атрибуты `code` и `name`, используя принцип обобщения. Все остальные сущности наследуются от него, добавляя специфические атрибуты через методы доступа (`set/get`), обеспечивающие инкапсуляцию данных.

2.1.1 Модуль `general.py`

Этот базовый класс обрабатывает общие атрибуты `code` и `name`, используя принцип обобщения.

Листинг 2.1: Модуль `general.py` — базовый класс сущностей

```
1 class general:
2     def __init__(self, code=0, name=''):
3         self.setCode(code)
4         self.setName(name)
5     def setCode(self, value): self.__code=value
6     def setName(self, value): self.__name=value
7     def getCode(self): return self.__code
8     def getName(self): return self.__name
```

2.1.2 Модуль `department.py`

Класс для сущности «Отдел» (`department`), наследуемый от `general`. Добавляет атрибут `employees` (количество сотрудников).

Листинг 2.2: Модуль `department.py` — сущность «Отдел»

```
1 from general import general
```

```

2
3 class department(general):
4     def __init__(self, code=0, name='', employees=0):
5         general.__init__(self, code, name)
6         self.setEmployees(employees)
7     def setEmployees(self, value): self.__employees=value
8     def getEmployees(self): return self.__employees

```

2.1.3 Модуль expensetype.py

Класс для сущности «Вид расхода» (expensetype), включающий описание и предельную норму.

Листинг 2.3: Модуль expensetype.py — сущность «Вид расхода»

```

1 from general import general
2
3 class expensetype(general):
4     def __init__(self, code=0, name='', description='', limit=0):
5         general.__init__(self, code, name)
6         self.setDescription(description)
7         self.setLimit(limit)
8     def setDescription(self, value): self.__description=value
9     def setLimit(self, value): self.__limit=value
10    def getDescription(self): return self.__description
11    def getLimit(self): return self.__limit

```

2.1.4 Модуль expense.py

Класс «Расход» (expense), который связывает отдел и вид расхода через агрегацию. Содержит поля amount (сумма) и date (дата), а также вспомогательные методы для получения названий связанных объектов.

Листинг 2.4: Модуль expense.py — сущность «Расход»

```

1 from general import general
2 from department import department
3 from expensetype import expensetype
4
5 class expense(general):
6     def __init__(self, code=0, name='', expensetype=None, department=None, amount
7         =0, date=''):
8         general.__init__(self, code, name)
9         self.setExpenseType(expensetype)
10        self.setDepartment(department)
11        self.setAmount(amount)
12        self.setDate(date)
13    def setExpenseType(self, value): self.__expensetype=value
14    def setDepartment(self, value): self.__department=value
15    def setAmount(self, value): self.__amount=value
16    def setDate(self, value): self.__date=value
17    def getExpenseType(self): return self.__expensetype
18    def getExpenseTypeCode(self):
19        if self.getExpenseType(): return self.getExpenseType().getCode()
20        else: return 0

```